PROJECT-BASED LEARNING

PHASE-II EVALUATION

Design and Implementation of a Skip List-Based Key-Value Store

Team ID: DSCPP-III-2026-T049

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T049
Semester: 3rd
Section: DS1, ML2, CYBER, ML1
Domain: DSCPP

Team Members

1. Sarthak Gera – 2510380077
2. Arjun Bhadana – 2510370135
3. Rudraksh Gera – 2510350038
4. Divyansh Silswal – 2510370989

Mentor

Ms. Priyanka Rastogi

Interactions completed:5

Phase-I Recap & Feedback Incorporated

Phase-I Status

Phase I proposed a templated C++ Skip List for ordered key-value operations.
The scope covered insertion, search, deletion, traversal, and inclusive range queries.

Feedback Received

- Demonstrate Skip List versatility through multiple real-world applications.
- Compare Skip List performance with an AVL Tree implementation.
- Visualize the randomized multilevel structure and traversal path.

Changes Incorporated

- Added leaderboard and stock order book demonstrations.
- Defined AVL implementation and benchmarking as Phase III work.
- Added level-wise display for visualizing promoted nodes.

Problem Statement & Finalized Objectives

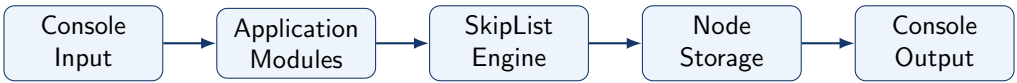
Problem Statement

Implementation of Skip List using efficient search, updates, deletion, and range queries without complex tree rotations.

Finalized Objectives

1. Implement a reusable templated Skip List with safe dynamic-memory cleanup.
2. Support expected $O(\log n)$ insertion, search, deletion, minimum, and maximum.
3. Demonstrate reuse through key-value, leaderboard, and order-book applications.
4. Validate correctness, then benchmark against AVL Tree and STL structures.

Updated System Architecture / Workflow



Major Components

- Generic `SkipList<K,V>` and `Node<K,V>` templates.
- Key-value, leaderboard, and order-book application modules.
- Menu-driven controller routes input and displays results.

Changes Since Phase-I

- Expanded one key-value store into three reusable applications.
- Added separate bid and ask lists with trade matching.
- Added level visualization, extrema queries, and sorted traversal.

Module Design & Functional Requirements

Major Modules

1. Generic Skip List Engine
2. Basic Key-Value Store
3. Game Leaderboard
4. Stock Order Book

Functional Requirements

- Perform CRUD operations and ordered display.
- Return ranges, minimum, and maximum keys.
- Rank players and preserve tied scores.
- Match orders and record partial fills.

Module Dependency

All applications depend on the shared templated Skip List engine.

Technology Stack & Development Methodology

Technology Stack

- Programming: C++17
- Database: None; in-memory storage
- Libraries: C++ Standard Library
- Development Tool: VS Code and g++
- Version Control: Git and GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	Linked lists, forward-pointer arrays, searching, traversal, and complexity analysis	Skip List Engine
OOPs with C++	Templates, classes, constructors, destructors, encapsulation, vectors, and maps	All Modules

Integration: One generic data structure supports three applications through **templates and modular OOP design**.

Development & Implementation Progress

Module	Status	Work Completed
Skip List Engine	Completed	Insert, update, search, delete, range, extrema, traversal, and cleanup
Application Demos	Completed	Key-value store, leaderboard, order matching, and trade history
Input Validation	In Progress	Positive-value checks exist; non-numeric stream recovery remains
Comparison Suite	Pending	AVL implementation and large-dataset benchmarks remain

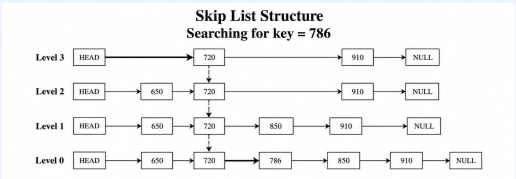
Overall Project Completion: 70%

Working Prototype / Demonstration

Implemented Features

- Ordered `SET`, `GET`, `DEL`, and `RANGE` operations.
- Level-wise visualization of randomized forward links.
- Leaderboard updates, ties, top ten, and score ranges.
- Order matching, partial fills, aggregation, and trade history.

Prototype / Output



View Complete Project Gallery

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Insert and update	Sorted keys; existing value replaced	Keys sorted; no duplicate created	Pass
Leaderboard ties	Equal scores remain grouped	Kabir and Neha grouped at 850	Pass
Non-numeric input	Reject input and re-prompt	`cin` remains failed; menu loops	Fail

Results / Validation

Five documented functional scenarios passed; one input-validation scenario failed.
Quantitative AVL and STL performance benchmarks remain pending.

GitHub Repository & Development Activity

Repository Information

- Repository: DSCPP_PBL_SEM_III
- README.md: Available
- Contributors: Four accounts
- Branch: `main`
- Integrated Basic Implementation, Leaderboard and Order Book demos into the project repository.

Development Activity

- Uploaded all Skip List source code, modules.
- Reports and modules are versioned.
- All the team members contribute to Development, Testing and Documentation.
- Latest commits update `README.md`.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Sarthak Gera	Team Lead, DSA Developer	Template, Random Structure Level, Leaderboard Design	12
Arjun Bhadana	DSA Developer	Basic Skip List implementation, leaderboard and traversal	14
Rudraksh Gera	Developer	Core operations, order book, extrema, repository and documentation	8
Divyansh Silswal	Testing, OOPs	Order book, console, template and testing	8

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Maintaining multilevel pointers.
- Reusing one engine for three applications.
- Handling ties and partial order fills.

Solutions / Learning

- Used predecessor arrays.
- Applied specialized template values.

Phase-III Roadmap

1. Recover from invalid console input.
2. Implement AVL Tree operations.
3. Benchmark against AVL and STL.
4. Optimize ranks using span information.
5. Finalize testing and documentation.

Expected Outcomes & References

Expected Final Outcomes

- Reusable Skip List with three applications.
- Benchmarked core operation performance.
- AVL Tree and STL comparison.
- Robust input and memory handling.
- Persistence-ready extensible architecture.

References

1. W. Pugh, "Skip Lists," *CACM*, 1990.
2. Redis Sorted Sets documentation.
3. LevelDB MemTable documentation.
4. cppreference: `map`, `unordered_map`.
5. GitHub: DSCPP_PBL_SEM_III.

Thank You
Questions & Suggestions